Voyage-AI — Implementation Status & Feature Inventory

Document Reference: `docs/reports/implementation-status.pdf` / `docs/reports/implementation-status.md`
System Version: 0.2.1 (Phase 2 Verified Baseline)
Author: Antigravity AI Engineering Team
Audit Date: October 25, 2026
Status: Implementation-Verified

---

1. Status Classification Definitions

Every component, feature, and subsystem in Voyage-AI is classified according to strict evidence-based criteria:

- **Implemented and verified:** The implementation exists in active source code and has been conclusively verified through passing automated test execution (0 failures, 0 errors) and live host runtime inspection.
- **Implemented but unverified:** The implementation exists in active source code, but automated tests or runtime verification have not been executed in this session.
- **Partially implemented:** Code exists for a portion of the feature, but the complete end-to-end operational workflow is incomplete.
- **Planned:** Formally specified in architecture or requirement documentation (`PRD.md` , `TDD.md` , `IMPLEMENTATION_PLAN.md`), but no active implementation code exists in the repository.
- **Deprecated or superseded:** Previously proposed, documented, or implemented, but explicitly replaced by newer architecture or discarded.

---

2. Comprehensive Implementation Status Inventory

Component or Feature	Status	Evidence (Source Files)	Verification Performed
Vite & React Application Shell	Implemented and verified	`frontend/src/App.tsx` , `frontend/src/main.tsx` , `frontend/package.json`	Executed `npm run test:unit` (11/11 passed)
Tactile Button Component	Implemented and verified	`frontend/src/components/common/Button.tsx`	Tested in `App.test.tsx` ; `:active:scale(0.97)` styling in `index.css`
Semantic Tag Badges	Implemented and verified	`frontend/src/components/common/Badge.tsx`	Tested in `App.test.tsx` ; renders regional, style, and license pills
Accessible Shimmer Skeletons	Implemented and verified	`frontend/src/components/common/SkeletonCard.tsx`	Tested in Vitest suite
Zero-Results Empty State	Implemented and verified	`frontend/src/components/common/EmptyState.tsx`	Tested in `App.test.tsx` ; verified filter reset action
Image Loader with Fallback	Implemented and verified	`frontend/src/components/common/ImageWithFallback.tsx`	Verified via `ImageWithFallback.test.tsx` (3/3 tests passed)
Editorial Navbar & Brand Header	Implemented and verified	`frontend/src/components/layout/Navbar.tsx`	Tested in `App.test.tsx` ; live health polling to `${apiBaseUrl}/health`
Editorial Hero Showcase	Implemented and verified	`frontend/src/components/discovery/HeroSection.tsx`	Tested in `App.test.tsx` ; WCAG AA contrast overlays
Multi-Attribute Filter Bar	Implemented and verified	`frontend/src/components/discovery/FilterBar.tsx`	Tested in `App.test.tsx` (region, style, budget filters, search)
Destination Card with Media License	Implemented and verified	`frontend/src/components/discovery/DestinationCard.tsx`	Tested in `App.test.tsx` ; climate badge, stay count, photographer credit link
5-Tab Detail Inspection Modal	Implemented and verified	`frontend/src/components/discovery/DestinationDetailModal.tsx`	Tested in `DestinationDetailModalTest.java` (ESC dismissal, tabs)
Server-Side Catalog Pagination	Implemented and verified	`frontend/src/App.tsx` , `DestinationController.java`	Verified in `DestinationsDiscoveryTest.java` (5/5 passed)
Backend Application Bootstrap	Implemented and verified	`backend/src/main/java/com/voyageai/VoyageAiApplication.java`	Bootstraps in `VoyageAiApplicationTests.java`
Uniform API Response Envelope	Implemented and verified	`backend/src/main/java/com/voyageai/common/api/ApiResponse.java`	Standardizes `{success, message, data, timestamp}`
Global Exception Handler	Implemented and verified	`backend/src/main/java/com/voyageai/common/api/GlobalExceptionHandler.java`	Verified in controller tests for 400, 401, 403, 404, 409, 500, 503

Component or Feature	Status	Evidence (Source Files)	Verification Performed
Service Health Check API	Implemented and verified	`backend/src/main/java/com/voyageai/common/health/HealthController.java`	Verified in `HealthControllerTest.java` and `HealthApiTest.java`
Spring Boot Actuator Health	Implemented and verified	`backend/pom.xml`, `backend/src/main/resources/application.yml`	Verified via `HealthApiTest.java`
Cross-Origin Security (CORS)	Implemented and verified	`backend/src/main/java/com/voyageai/config/CorsConfig.java`	Verified cross-origin calls between :5173 and :8081
Stateless JWT Bearer Auth	Implemented and verified	`com.voyageai.security.*`, `com.voyageai.auth.*`	Verified via `AuthApiTest.java` (10/10 passed)
User Roles & Profile Persistence	Implemented and verified	`User.java`, `UserController.java`, `UserService.java`	Verified via `UserApiTest.java` and `UserProfileApiTest.java`
Open-Meteo Weather Service	Implemented and verified	`backend/src/main/java/com/voyageai/integration/weather/OpenMeteoWeatherService.java`	Verified in `OpenMeteoWeatherServiceTest.java` (6/6 passed)
Weather REST Controller	Implemented and verified	`backend/src/main/java/com/voyageai/integration/weather/WeatherController.java`	Verified via `WeatherControllerTest.java` (7/7 passed)
Google Gemini AI Integration	Implemented and verified	`backend/src/main/java/com/voyageai/integration/ai/GoogleGeminiService.java`	Verified via `AiControllerTest.java` (6/6) & `AiApiTest.java` (3/3)
AI Travel Assistant Component	Implemented and verified	`frontend/src/components/ai/AiTravelAssistant.tsx`	Verified via `AiTravelAssistant.test.tsx` (4/4 passed)
Database Migrations (Flyway V0-V4)	Implemented and verified	`V0` through `V4` SQL migration scripts in `db/migration/`	Verified via `DatabaseFlywayIntegrationTest.java` (4/4 passed)
Canonical MySQL Database (`voyage`)	Implemented and verified	`infra/docker-compose.yml`, `backend/src/main/resources/application.yml`	Verified live on host port 3307
Domain Relational Persistence	Implemented and verified	`User`, `Destination`, `TravelListing`, `Trip`, `TripDay`, `ItineraryItem`	Verified via `PersistenceIntegrationTest.java` (3/3 passed)
Destination Discovery APIs	Implemented and verified	`com.voyageai.destination.*`, `com.voyageai.listing.*`	Verified via `DestinationApiTest.java` (9/9 passed)
Real Wikimedia Ingestion Pipeline	Implemented and verified	`DestinationIngestionService.java`, `AdminDestinationController.java`	Verified via `DestinationIngestionServiceTest.java` (13/13 passed)
Admin Authorization Guard	Implemented and verified	`AdminDestinationController.java`, `SecurityConfig.java`	Verified via `AdminDestinationControllerTest.java` (3/3 passed)
Trip & Itinerary Management APIs	Implemented and verified	`com.voyageai.trip.*`, `com.voyageai.itinerary.*`	Verified via `TripApiTest.java` (8/8) & `PersistenceTripApiTest` (3/3)
Database Isolation Guard	Implemented and verified	`backend/src/test/java/com/voyageai/integration/db/DatabaseFlywayIntegrationTest.java`	Executed during `mvn test`
Docker Compose Orchestration	Implemented and verified	`infra/docker-compose.yml`, `infra/init-db.sql`	Verified via `docker compose up -d`
Local Launcher (`start.bat`)	Implemented and verified	`start.bat` in repository root	Executed and validated via Windows `cmd.exe`
Interactive Leaflet Map Tiles	Planned (Phase 3 Target)	Specified in `docs/DATA_AUTHENTICITY_AND_PROVIDER_PLAN.md`	Policy verified against OSM usage guidelines
OSRM Transit Distance Routing	Planned (Phase 4 Target)	Specified in `docs/DATA_AUTHENTICITY_AND_PROVIDER_PLAN.md`	Abstraction designed; integration pending

3. Comprehensive Verification Summary

- **Total Backend Tests: 83 passed, 0 failures, 0 errors, 0 skipped** (`backend/pom.xml`).
- **Total Frontend Tests: 11 passed, 0 failures, 0 skipped** (`frontend/package.json`).
- **Total Browser & Integration Tests: 34 passed, 0 failures, 0 skipped** (`tests/pom.xml`).
- **Combined Test Total: 128 passed, 0 failures, 0 skipped (100% pass rate)**.
- **Flyway Migrations:** All 5 migrations (`V0__baseline` , `V1__init_schema` , `V2__seed_travel_data` , `V3__user_profile_and_destination_indexes` , `V4__add_user_roles_and_image_licensing`) apply cleanly and idempotently to the canonical database `voyage` .
- **Live Google Gemini Verification:** Verified via both direct REST API and automated Playwright browser test using prompt `"What's capital of India?"` → returned `"New Delhi"` from `gemini-3.5-flash-lite` .
- **Live Open-Meteo Weather Verification:** Verified live coordinate queries returning real temperature and weather condition metrics with CC-BY 4.0 provenance.
- **Real Wikimedia Ingestion:** Verified real ingestion from English Wikipedia and Wikimedia Commons Action API with CC-BY-SA 4.0, CC0, and Public Domain licensing and photographer attribution.
- **Zero-Fabrication Policy:** Maintained across all destination data, listings, weather queries, and AI generation.
- **Detailed Verification Report:** See POST_IMPLEMENTATION_VERIFICATION.md for full traceability and performance logs.